

BUKU SAKU SURVIVAL SPRING BOOT UNTUK PELATIHAN & SERTIFIKASI

1. Gambaran Besar Teknologi

Saat membuat aplikasi backend menggunakan Spring Boot, alurnya biasanya:

Client (Browser/Postman) ↓ Controller ↓ Service ↓ Repository ↓ MySQL Database

Contoh:

POST /products

Controller menerima request ↓ Service memproses logika ↓ Repository menyimpan ke database ↓ Database menyimpan data

2. JAVA OOP (OBJECT ORIENTED PROGRAMMING)

Apa itu Class?

Class adalah blueprint atau cetakan.

Contoh:

Mobil memiliki:

- warna
- merk
- kecepatan

Maka dibuat class:

```
public class Mobil {  
  
    String warna;  
    String merk;  
    int kecepatan;  
  
}
```

Apa itu Object?

Object adalah hasil pembuatan dari class.

```
Mobil mobilSaya = new Mobil();
```

Sekarang mobilSaya adalah object.

Constructor

Digunakan saat object dibuat.

```
public class User {  
  
    String nama;  
  
    public User(String nama){  
        this.nama = nama;  
    }  
  
}
```

Pemakaian:

```
User user = new User("Naila");
```

Encapsulation

Data dibuat private.

```
public class User {  
  
    private String nama;  
  
    public String getNama(){  
        return nama;  
    }  
  
    public void setNama(String nama){
```

```
        this.nama = nama;
    }
}
```

Inheritance

Turunan dari class lain.

```
public class Hewan {

    public void makan(){
        System.out.println("Makan");
    }

}
```

```
public class Kucing extends Hewan {

}
```

Polymorphism

Method yang sama bisa berbeda perilaku.

```
public class Hewan {

    public void suara(){
        System.out.println("Suara hewan");
    }

}
```

```
public class Kucing extends Hewan {

    @Override
    public void suara(){
        System.out.println("Meong");
    }

}
```

```
}
```

3. STRUKTUR PROJECT SPRING BOOT

Biasanya:

src/main/java

|—— controller |—— service |—— repository |—— entity |—— dto |—— DemoApplication

Penjelasan:

Controller → menerima request

Service → logika bisnis

Repository → akses database

Entity → representasi tabel

DTO → data request/response

4. REST API

REST API adalah cara frontend berbicara dengan backend.

GET

Mengambil data

```
GET /products
```

POST

Menambah data

POST /products

PUT

Mengubah data

PUT /products/1

DELETE

Menghapus data

DELETE /products/1

5. CONTROLLER

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping
    public String getAllProducts(){
        return "Data Product";
    }

}
```

Akses:

http://localhost:8080/products

6. DATABASE MYSQL

Membuat database

```
CREATE DATABASE training_db;
```

Melihat database

```
SHOW DATABASES;
```

Menggunakan database

```
USE training_db;
```

7. ORM & JPA

ORM = Object Relational Mapping

Mengubah object Java menjadi data database.

Contoh Entity:

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private Double price;
}
```

8. REPOSITORY

```
@Repository
public interface ProductRepository
extends JpaRepository<Product, Long> {
```

```
}
```

JpaRepository otomatis menyediakan:

save()

findAll()

findById()

deleteById()

9. SERVICE

```
@Service
public class ProductService {

    @Autowired
    ProductRepository repository;

    public List<Product> getAll(){
        return repository.findAll();
    }

}
```

10. EXCEPTION HANDLING

Contoh:

```
throw new RuntimeException("Data tidak ditemukan");
```

Global Handler:

```
@ControllerAdvice
public class GlobalExceptionHandler {
```

```
}
```

11. FILE UPLOAD

```
@PostMapping("/upload")
public String upload(
    @RequestParam MultipartFile file){

    return file.getOriginalFilename();

}
```

12. SECURITY

Tujuan:

- Login
- Proteksi endpoint

Contoh:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

}
```

13. TESTING JUNIT

```
@Test
void testTambahData(){

}
```

Tujuan:

Memastikan program berjalan sesuai harapan.

14. LATIHAN PRAKTIK YANG HARUS DICoba

TARGET: Membuat CRUD Product.

Field:

id name price

STEP 1

Buka IntelliJ

New Project

Spring Boot

Pilih:

Java Maven Java 17

Dependency:

Spring Web Spring Data JPA MySQL Driver Lombok

STEP 2

Buat database

DBeaver

Jalankan:

```
CREATE DATABASE training_db;
```

STEP 3

Isi application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/training_db
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

STEP 4

Buat Entity Product

```
@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private Double price;

}
```

STEP 5

Buat Repository

```
public interface ProductRepository
extends JpaRepository<Product, Long> {

}
```

STEP 6

Buat Service

```
@Service
public class ProductService {

    @Autowired
    ProductRepository repository;

    public List<Product> getAll(){
        return repository.findAll();
    }

}
```

STEP 7

Buat Controller

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    ProductRepository repository;

    @GetMapping
    public List<Product> getAll(){
        return repository.findAll();
    }

}
```

STEP 8

Jalankan aplikasi

Klik tombol Run

Jika berhasil akan muncul:

Started DemoApplication

STEP 9

Buka Browser

<http://localhost:8080/products>

STEP 10

Buka Postman

Method:

POST

Body JSON:

```
{  
  "name": "Keyboard",  
  "price": 500000  
}
```

Kirim ke:

<http://localhost:8080/products>

CHECKLIST SIAP PELATIHAN

☐ Java 17 terinstall

☐ Maven berjalan

☐ IntelliJ berjalan

☐ Laragon MySQL berjalan

☐ DBeaver connect ke MySQL

☐ Postman berjalan

☐ Bisa membuat project Spring Boot

☐ Bisa menjalankan localhost:8080

☐ Paham GET POST PUT DELETE

☐ Bisa membuat Entity sederhana

Jika semua checklist ini sudah terpenuhi, kemungkinan besar kamu tidak akan kesulitan mengikuti pelatihan dan studi kasus dasar Spring Boot.